FinAI

An AI-Driven Personal Finance, Investment Analytics and Financial Intelligence Platform

M.TECH DISSERTATION · INTERNAL REVIEW 1

Phase 1 - Foundation & Core Financial Management

September 2026

REVIEW 1

Agenda

01	Problem statement & existing system limitations
02	Research gap and proposed system objectives
03	System architecture & technology stack
04	Database, API and security design
05	Phase 1 AI module: anomalies, insights, health score
06	UI prototype, implementation status & test results
07	Limitations and the Phase 2 roadmap

WHY THIS PROJECT

Problem Statement

Existing tools record financial life but never explain it. Scores are black boxes, investment analysis is divorced from daily behaviour, and AI output - where it exists - is unsupported by the user's actual data.

Passive tracking

Apps display spending but never interpret change, cause or action.

Black-box scores

A number appears with no factors, weights or arithmetic to verify.

Ungrounded AI

Generic tips that no dataset supports - hallucination, not analysis.

Target: an integrated, explainable, data-grounded financial intelligence platform.

LITERATURE SURVEY

Existing System Limitations

Spreadsheet budgets	Manual, error-prone, zero automation or anomaly detection
Bank / UPI apps	Capture transactions but only record - no interpretation
Expense trackers	Charts without meaning: no guards, no evidence, no scoring
Investment trackers	Detached from daily spending; opaque return metrics
Robo-advisory tools	Black-box recommendations; advice, not education

FROM SURVEY TO PROPOSAL

Research Gap → FinAI Answer

1	**Tracking without explanation**	→	Guarded insight engine: every insight cites its evidence
2	**Black-box scores**	→	Weighted health score with visible per-factor contributions
3	**No integrated research**	→	Stock analyzer + valuation + sentiment (Phase 2, provider-abstracted)
4	**AI without grounding**	→	RAG assistant with mandatory citations; refuses on empty retrieval (P4)
5	**Planning divorced from behaviour**	→	Goals & calculators consume the user's real expense history (P3)

WHAT PHASE 1 DELIVERS

Proposed System & Objectives

Objectives

- Secure layered architecture with RBAC and versioned disclaimer governance
- Expense, income and budget management with threshold alerts
- Statistically grounded AI: anomaly detection, guarded insights, transparent scoring
- Measurable evaluation for every model (precision / recall / F1 / MAPE)
- Phase 2-4: investment analytics → portfolio intelligence → RAG assistant

Explicitly out of scope

No F&O recommendations

Derivatives trading is outside platform scope - higher complexity and risk. Stated in the product itself.

Not investment advice

Analytical and educational only. A versioned risk disclaimer must be accepted before use; acceptance (version, IP, time) is stored.

47	tests passing	35	REST endpoints
8	DB entities	4	dev phases

LAYERED DISCIPLINE

System Architecture

Browser SPA — React 18 + TypeScript + Tailwind - glassmorphism dashboard

↓

API routes — FastAPI · validation · JWT auth · RBAC · rate limiting

↓

Services — Business rules: auth, expenses, budgets, analytics, admin, audit

↓

Analysis engine — Pure functions - aggregation · anomaly · insights · scoring (no I/O)

↓

Data access — SQLAlchemy 2.0 models + session scoping

PostgreSQL (prod) / SQLite (dev) · Redis + external providers reserved for Phase 2+

Layer rules

- **Routes** never contain business logic
- **Services** never touch HTTP constructs
- **Engine** pure computation - unit-testable without I/O
- **Data** no business rules

Startup waits for database readiness; /healthz reports app + DB status.

CHOICES WITH REASONS

Technology Stack & Justification

Backend	**FastAPI + Pydantic v2**	Async, typed contracts, auto-OpenAPI; fits Python ML ecosystem
Database	**PostgreSQL 16 / SQLite dev**	One SQLAlchemy schema - zero-setup laptop demo, production-grade deploy
Auth	**JWT 30 min + rotating refresh, bcrypt**	Revocable hashed sessions; no plaintext credentials
Frontend	**React 18 + TypeScript + Vite**	Private dashboard: no SSR benefit; fast builds, strict types
UI / charts	**Tailwind 3 + Recharts**	Consistent glass design system; light chart composition
Testing	**pytest + TestClient**	Isolated DB per test; public-API-driven integration
Deploy	**Docker Compose**	postgres + redis + api + nginx web - production-shaped

PHASE 1 ENTITIES

Database Design

users	session_tokens	disclaimer_acceptances
password_reset_tokens	expense_categories	expenses
incomes	budgets	audit_logs

Design decisions

- **Money = Numeric(14,2) -** floating point never touches currency
- **Soft deletion -** analytics history survives corrections
- **Token digests only -** HMAC-SHA256 of refresh tokens; a DB leak yields nothing usable
- **Composite indexes -** (user_id, expense_date) · (user_id, category_id) · token_hash unique
- **System vs custom categories -** system rows shared, deletion-guarded at service layer

JWT + ROTATING REFRESH

Authentication Architecture

1

Login

bcrypt verify; 5 failures → 15 min lock; audit-logged

2

Token pair

access JWT 30 min (jti) + refresh 14 days stored as HMAC digest

3

Refresh rotates

used token revoked & replaced - replay fails fast

4

Sessions

list devices, revoke any; logout revokes server-side

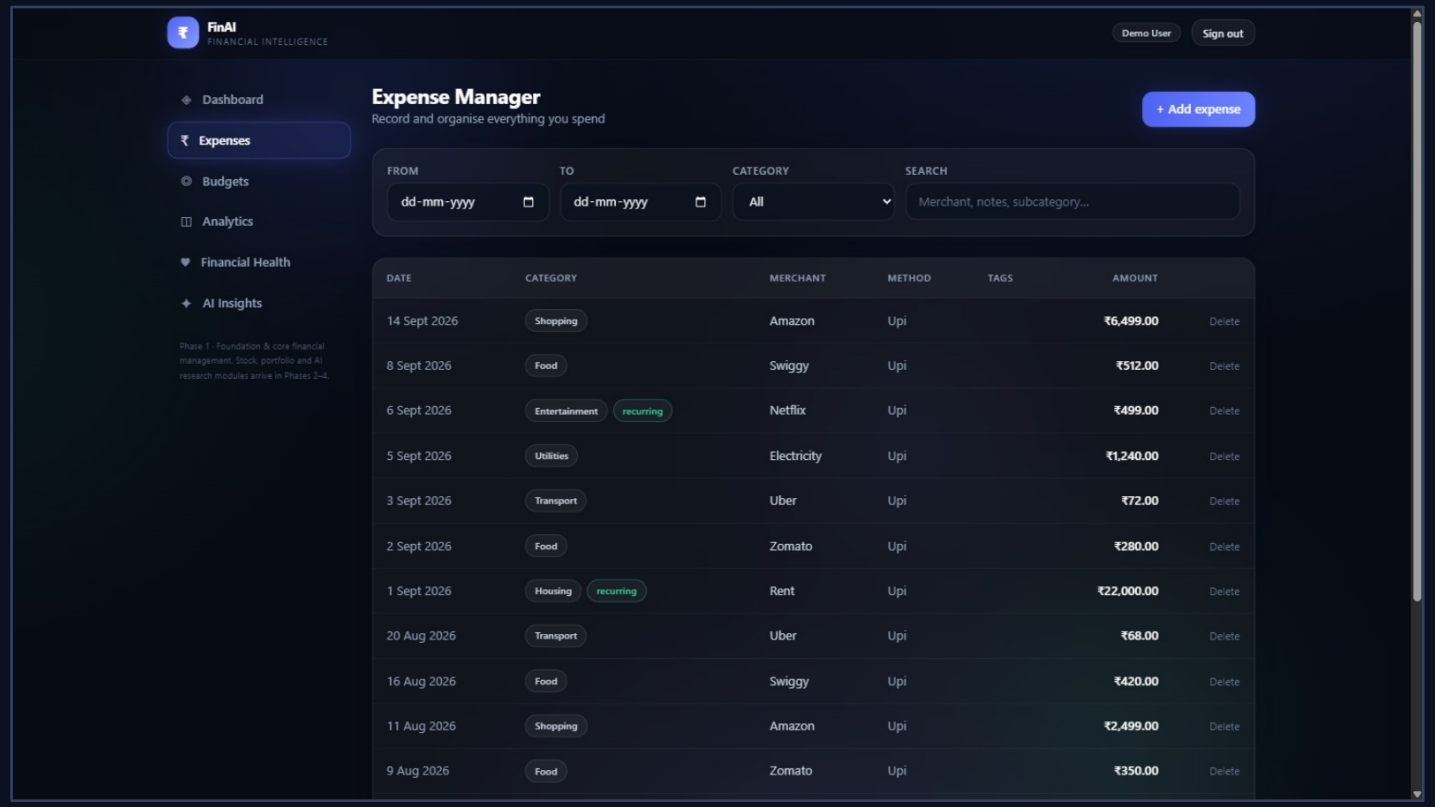
Disclaimer governance & hardening

- **Versioned disclaimer** acceptance (version, IP, timestamp) persisted; login blocked until current version accepted
- **Brute force** per-IP rate limit 10/5 min + account lockout + failure audit
- **Reset flow** single-use expiring hashed tokens; dev-only visibility; all sessions revoked on reset
- **Transport** Authorization-header tokens only - no cookie CSRF surface; nosniff, DENY, no-store headers

MODULE WALKTHROUGH

Expense & Budget Management

- **Full metadata** date, amount, category/subcategory, payment method, merchant, tags, notes
- **Recurring flags** feed the recurring-load insight and recurring-ratio score
- **Custom categories** 12 system categories shared; user categories deletion-guarded
- **Budgets** monthly/yearly, overall or per category, 50/75/90% alert chain
- **Soft delete + audit** history preserved; every change recoverable



Expense manager - filters, search, pagination, custom categories

Budget screens: live utilisation bars, remaining amounts and triggered alerts.

Z-SCORE WITH GUARDS

AI Module · Anomaly Detection

$$z = (x - \mu) / \sigma \qquad \text{flag when } z > 2.5$$

Two detectors: per-transaction within category · daily totals against user baseline

≥ 8 samples

per category before any transaction flag

≥ 14 days

of history before daily-total detection

μ, σ, z, n reported

every flag carries its reference statistics and a plain-language explanation

Silence over invention

thin data produces zero flags, by design

Every flag: amount · mean · std · z-score · sample size · explanation

SIX RULES, EACH CITING EVIDENCE

AI Module · Guarded Insight Engine

Category spike

fires when > 1.2× previous month and +₹500

“Food up 185% - ₹2,198 vs ₹770.”

Concentration

fires when one category > 35% of spend

“Housing is 71% of this period.”

Recurring load

fires when ≥ 3 merchants over ≥ 3 months

“Netflix + Spotify ≈ ₹798/month.”

Weekend pattern

fires when weekend avg > 1.3× weekday, ≥ 14 days

“Weekends cost 1.6× weekdays.”

Savings trend

fires when rate change > ±5 pp, 2 months

“Savings rate fell 8 points.”

Budget pressure

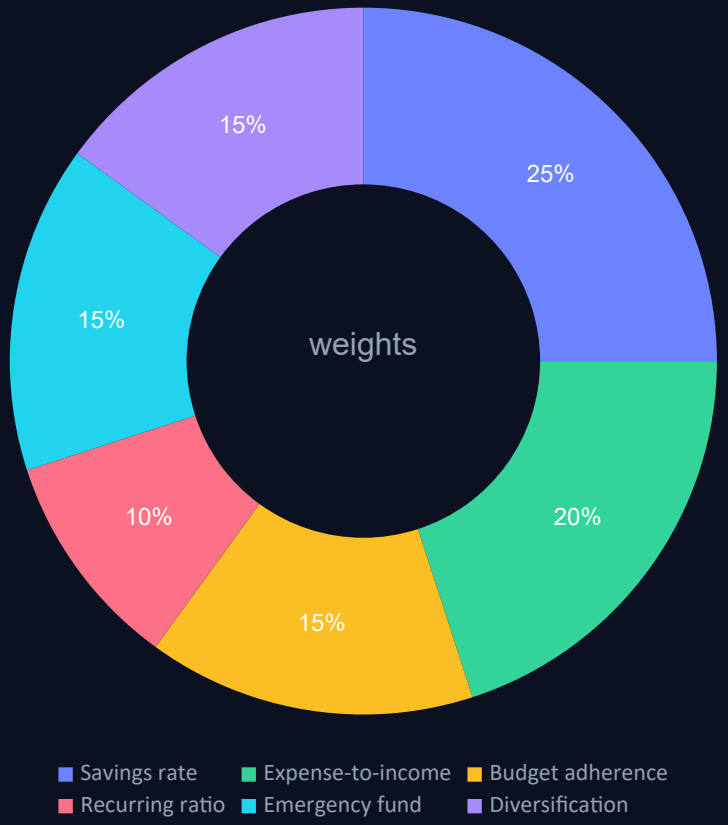
fires when utilisation crosses threshold

“Food budget exceeded - 146%.”

Guard fails → rule stays silent. Guard passes → insight + evidence object with the exact figures.

TRANSPARENT WEIGHTED MODEL

Explainable Financial Health Score



Score = Σ factorScore × effectiveWeight

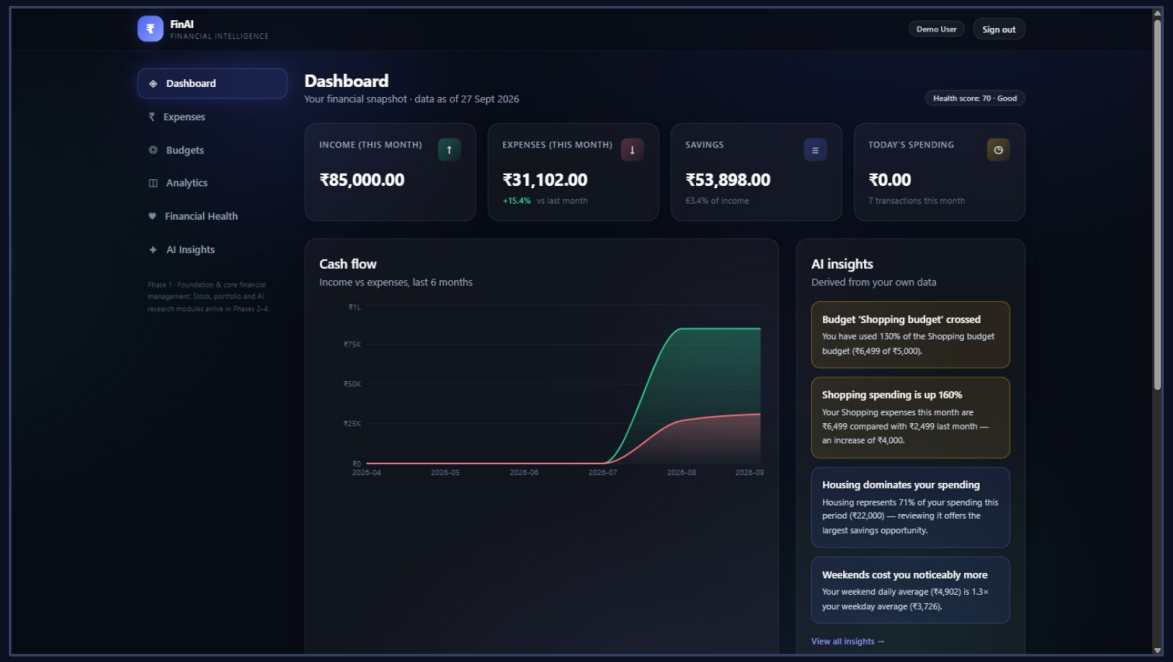
Factors without data are marked not_evaluated and their weight is redistributed across evaluated ones - the model never invents inputs. Response exposes every weight, factor score and contribution.

85+	70+	55+	40+	<40
Excellent	Good	Fair	Needs Work	At Risk

Live demo output: score 70 (Good) - savings 100 · expense ratio 100 · budget 0 · recurring 0 · emergency 87 · diversification not evaluated (Phase 3)

GLASSMORPHISM DESIGN SYSTEM

UI / UX Prototype



Dashboard - snapshot, cash-flow, insights, budgets



Financial health - every factor, weight and contribution

- Responsive desktop-first, mobile-ready
- Dark ambient aurora theme, glass cards
- Keyboard-accessible forms, ARIA labels
- Interactive Recharts: donut · bar · line · area

DEFENCE IN DEPTH

Security Design

Authentication

bcrypt (cost 12) · JWT +jti · rotating digest-stored refresh · lockout · rate limits 10/5min auth, 120/min mutations

Authorization

RBAC: users see only their data; admin sees operational counts - never user financial records

Input & injection

Pydantic schemas on every body · bound-parameter SQL only · React escaping · security headers on all responses

Audit & observability

append-only trail: logins, failures, resets, admin actions · suspicious-activity flag on failure bursts

Known Phase 1 limits, tracked openly: dev-only reset tokens · per-process limiter (Redis in P4) · verification stubbed

STATUS

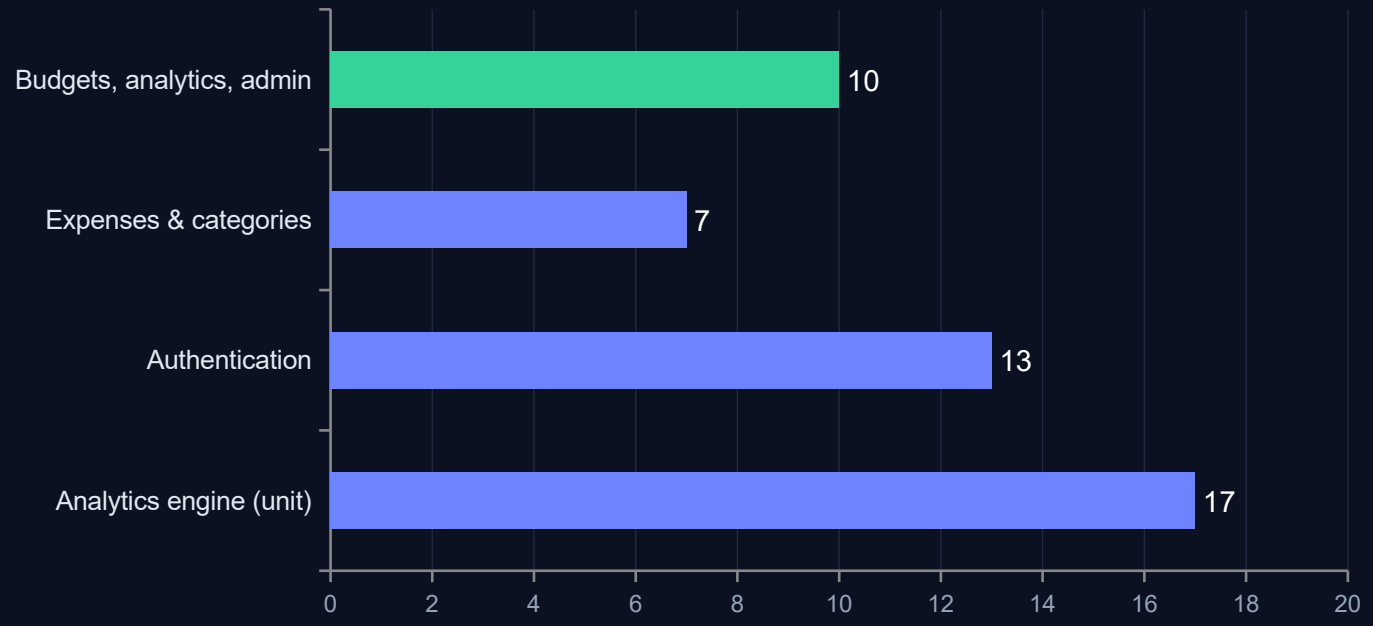
Current Implementation & Testing

47
backend tests, all passing

35
documented REST endpoints

9
UI modules delivered

13
architecture documents



Test distribution by suite (47 total)

Test strategy

- Unit: pure analytics engine, no I/O
- Integration: isolated DB per test
- Seeding via public API only
- pytest.approx for all money math
- RBAC + isolation explicitly tested

SCRIPTED END-TO-END JOURNEY ON THE DOCKER STACK

Initial Results

Disclaimer → Register → Income → 7 expenses → Budget → Dashboard → Insights → Health → RBAC 403 → Admin views

ALL CHECKS PASSED - on SQLite (dev) and PostgreSQL (containerised production build)

Insights	2 evidence-backed insights generated: budget pressure + category concentration
Budget alerts	94.1% utilisation triggered the 50 / 75 / 90 alert chain correctly
Health score	95 (Excellent); 5 of 6 factors evaluated - diversification correctly not evaluated
RBAC	user forbidden on admin endpoints (403); admin overview + audit trail verified

HONEST SCOPE

Limitations & Phase 2 Roadmap

Current limitations

- Market panels show “Data unavailable” - no fabrication before Phase 2
- Rate limiter per-process; Redis backing arrives in Phase 4
- Password reset via dev tokens until SMTP is wired
- Forecasting waits for sufficient history (guards refuse thin data)

Documented, not hidden - see risk register (14 items).

Phase 2 - Investment Analytics

- **Provider abstraction** FinancialDataProvider interface + cache; no site is load-bearing
- **Stock analyzer** fundamentals, statements, SMA/EMA · RSI · MACD · Bollinger
- **Sentiment (Exp. 2)** rules vs TF-IDF+SVM vs transformer - precision / recall / F1
- **Valuation engine** DCF with sensitivity grids · PE · PEG · EV/EBITDA · DDM
- **Scoring + reports** explainable stock score; PDF research reports; Alembic begins

Phase 1: complete, tested, deployed.

47 passing tests · full end-to-end journey on the containerised stack · every AI output explainable and evidence-backed

Next: Phase 2 - Investment Analytics

Provider abstraction · stock analyzer · valuation · sentiment experiment · PDF reports

Thank you · Questions & demonstration ready